

Django Views, Requests & Routing

Understanding view functions, URL configurations, HTTP methods, parameters, and error handling.

1. Fundamentals of Views

A **View** in Django is a callable Python function or class that accepts a Web/HTTP Request object and returns an HTTP Response object (such as HTML content, a redirect, or a 404 error).

Function-Based Views (FBVs)

Written as simple Python functions. Easy to read, transparent, and explicit. Excellent for handling custom linear logic.

Class-Based Views (CBVs)

Written as Python classes inheriting from Django generic views. Encourages code reusability (DRY) using object-oriented techniques.

2. HttpRequest and HttpResponse Objects

Whenever a client requests a page, Django creates an `HttpRequest` object containing metadata about the incoming call.

Object Attribute / Method	Description
<code>request.METHOD</code>	Returns the HTTP verb used in the request (e.g., <code>'GET'</code> , <code>'POST'</code>).
<code>request.GET</code>	A dictionary-like object containing all HTTP GET query parameters (<code>?key=value</code>).
<code>request.POST</code>	A dictionary-like object containing form submission data sent via POST.
<code>request.headers</code>	Access request headers like User-Agent, Accept-Language, or Content-Type.
<code>HttpResponse("text")</code>	Returns a plain text or HTML string directly to the client browser.
<code>JsonResponse(dict)</code>	Subclass of <code>HttpResponse</code> that automatically serializes Python dictionaries into JSON format.

3. Basic View & Response Example

```
from django.http import HttpResponse, JsonResponse
def home_view(request):
    if request.method == 'GET':
        return HttpResponse("<h1>Welcome to Little Lemon Restaurant</h1>")
    def api_menu(request):
        items = {"menu": ["Pizza", "Pasta", "Salad"]}
        return JsonResponse(items)
```

4. URL Configurations & Path Matching

Django routes incoming requests by matching the requested URL path against patterns defined in `urls.py`. URL routing is usually modularized into project-level and app-level files using `include()`.

```
# myproject/urls.py (Project-level URL conf)
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('restaurant/', include('myapp.urls')), # Includes app urls
]
```

5. URL Parameters & Converters

Path converters capture dynamic segments from a URL and pass them as keyword arguments to the view function.

Path Converter	Matching Type	Example URL Pattern
<code><str:name></code>	Matches any non-empty string excluding path separators (/). Default converter.	<code>path('user/<str:username>/', views.profile)</code>
<code><int:id></code>	Matches zero or any positive integer. Returns an <code>int</code> .	<code>path('menu/<int:pk>/', views.menu_detail)</code>
<code><slug:slug></code>	Matches letters, numbers, hyphens, and underscores.	<code>path('posts/<slug:title>/', views.post_detail)</code>
<code><uuid:id></code>	Matches a formatted UUID string (e.g., 075194d3...).	<code>path('order/<uuid:order_id>/', views.order_status)</code>

6. Query Parameters vs Dynamic Path Parameters

Dynamic Path Segment

URL: `/menu/item/42/`

Pattern: `path('menu/item/<int:id>/', views.detail)`

Passed as a argument: `def detail(request, id):`

Query String Parameter

URL: `/menu/?category=dessert`

Pattern: `path('menu/', views.menu_list)`

Accessed via GET dict:

`request.GET.get('category')`

7. URL Namespacing and Reverse Lookup

Namespacing prevents URL collisions across apps and enables dynamic URL generation:

```
# myapp/urls.py
app_name = 'myapp'
urlpatterns = [
    path('menu/', views.menu_list, name='menu_list'),
]
# In python view / template:
# reverse('myapp:menu_list') -> returns '/restaurant/menu/'
```

8. Handling HTTP Errors in Views

Django provides built-in mechanisms and exceptions to manage standard HTTP error codes gracefully.

Status Code	Meaning	Django View / Exception Handling
404 Not Found	Requested resource does not exist.	Raise <code>Http404("Message")</code> or use <code>get_object_or_404()</code> helper.
400 Bad Request	Invalid client request parameters or payload.	Return <code>HttpResponseBadRequest("Invalid data")</code> .
403 Forbidden	Client lacks authorization for the resource.	Raise <code>PermissionDenied</code> exception.
500 Server Error	Unhandled server exception or code crash.	Handled by Django's default handler or custom <code>handler500</code> view.

```
from django.shortcuts import get_object_or_404 from .models import MenuItem def
menu_item_detail(request, pk): # Automatically raises HTTP 404 if object with primary key does
not exist item = get_object_or_404(MenuItem, pk=pk) return render(request, 'menu_detail.html',
{'item': item})
```

9. Class-Based Views (CBVs)

Class-Based Views offer alternative view implementations utilizing inheritance and mixins instead of standard functions. They handle HTTP methods via dedicated class methods (`get()` , `post()`).

```
from django.views import View from django.http import HttpResponse class ContactView(View):
def get(self, request): # Handles HTTP GET requests return HttpResponse("Render contact form")
def post(self, request): # Handles HTTP POST requests return HttpResponse("Process form
submission") # In urls.py: # path('contact/', ContactView.as_view(), name='contact')
```

Key Takeaway: Function vs. Class-Based Views

Use **Function-Based Views (FBVs)** when logic is simple, straightforward, or custom. Use **Class-Based Views (CBVs)** when reusing common patterns like forms, CRUD operations, or list pagination.